

Program 1

```
#include <iostream>
using namespace std;

// Creating a node
class Node {                                //1|next -> 2|next -> 3|NULL
public:

};

int main() {
    Node* head;
    Node* one = NULL;

    // allocate 3 nodes in the heap , using new
    one = new Node();

    // Assign value values
    one->value = 1;

    // Connect nodes
    one->next = two;

    // print the linked list value
    head = one;
    while (head != NULL) {
        cout << head->value;
        head = head->next;
    }
}
```

Program 2

```
#include <iostream>
using namespace std;

class Node {
public:

};

class TheList {
```

```

private:

public:

    TheList() //constructor for TheList
    {
        head = NULL;
    }

    void insert(char data) { //insert at front

    }

    void display() {

        }
        cout << endl;
    }
};

int main() {
    TheList myList;
    myList.insert('Z');
    myList.insert('T');
    myList.insert('A');

    cout << "The Link List - Singly: ";
    //A|next -> T|next -> Z|NULL

    myList.display();

    return 0;
}

// MCQ

// What will be the output of the following code snippet for 10->11->12->13->14?
/*
void solve (ListNode* head) {
    while(head != NULL) {
        cout << head -> data << " ";
        head = head -> next;
    }
}

```

A. 10 11 12 13 14
 B. 11 12 13 14
 C. 14 13 12 11 10
 D. 13 12 11 10 */

Program 3

```
// C++ Program for Implementation of linked list
#include <iostream>
using namespace std;

// class for a node in the linked list
class Node {
    public :
};

// Define the linked list class
class LinkedList {
    // Pointer to the first node in the list

public:
    // Constructor initializes head to NULL

    // Function to Insert a new node at the beginning of the list
    void insertAtBeginning(int value) {

    }

    // Function Insert a new node at the end of the list
    void insertAtEnd(int value) {

    }

    // Function to Insert a new node at a specific position in the list
    void insertAtPosition(int value, int position) {

    }

    // Function to Delete the first node of the list
    void deleteFromBeginning() {

    }

    // Function to Delete the last node of the list
    void deleteFromEnd() {

    }

    // Function to find the key node in the list
    bool find(int key) {

    }

    // Function to Delete a node at a specific position in the list
    void deleteFromPosition(int position) {

    }
}
```

```

        // Function to print the nodes of the linked list
        void display() {

            }

}; //TheList

int main() {
    // Initialize a new linked list
    LinkedList list1;

    // Insert elements at the end
    list1.insertAtEnd(10);
    list1.insertAtEnd(20);

    // Insert element at the beginning
    list1.insertAtBeginning(5);

    // Insert element at a specific position
    list1.insertAtPosition(15, 3);

    cout << "Linked list after insertions: ";
    list1.display();

    bool f = list1.find(30);
    if (f==true )
        cout<<" Found "<<endl;
    else
        cout<<" Not Found "<<endl;

    // Delete element from the beginning
    list1.deleteFromBeginning();
    cout << "Linked list after deleting from beginning: ";
    list1.display();

    // Delete element from the end
    list1.deleteFromEnd();
    cout << "Linked list after deleting from end: ";
    list1.display();

    // Delete element from a specific position
    list1.deleteFromPosition(2);
    cout << "Linked list after deleting from position 2: ";
    list1.display();

    return 0;
}

```